Grama Jyothi Rural Electricity Co-operative Billing System

CO1 Assignment 1 — Complete Technical Analysis, C Implementation, and Verification Report

NAME: **Ginni Raghava**
REGISTER NO:**1925260329**

Task 1 — Understand the Problem and Identify the Data

Grama Jyothi Rural Electricity Co-operative currently bills 4,000 village households monthly, with planned expansion to a taluk (60,000 households) and a full district (800,000 households). The legacy spreadsheet suffers from three critical defects: floating-point representation errors causing discrepancies of several paise, negative consumption bills when 16-bit electromechanical/digital meter counters roll over from 65535 to 0, and duplicated literal tariff constants that cause partial updates during tariff revisions. Storing all monetary values in integer paise and modeling meter counters with 16-bit unsigned modular arithmetic eliminates all three failure modes.

Data Item	Role	Mutability	Sign / Domain Properties & Justification
household_id	Input	Variable per record	Strictly positive integer (*1* to *800,000* at district scale). Never negative or fractional.
category	Input	Variable per record	Discrete code ('D' Domestic, 'C' Commercial, 'A' Agricultural). Stored as single character or enumeration.
previous_reading	Input	Variable per record	Non-negative integer in *[0, 65535]* representing cumulative kWh on a 16-bit hardware register.
current_reading	Input	Variable per record	Non-negative integer in *[0, 65535]*. May be numerically smaller than previous_reading upon counter wraparound.
units_consumed	Output (Derived)	Computed per bill	Non-negative integer in *[0, 65535]*. Computed via modulo-*65536* unsigned subtraction.
fixed_charge_paise	Output / Constant	Constant during run	5000 paise (Rs 50.00). Fixed per billing cycle unless revised by the board.

energy_charge_paise	Output (Derived)	Computed per bill	Non-negative integer paise. Exact sum of slabwise unit charges; avoids fractional rupees (e.g., Rs 2.25 = 225 paise).
total_bill_paise	Output (Derived)	Computed per bill	Non-negative integer paise (≥ *5000* paise). Sum of fixed charge and energy charge.
district_total_paise	Output (Accumulator)	Updated per bill	Very large non-negative integer. At 800,000 houses averaging Rs 500 (50,000 paise), reaches 4×10^{10} paise, exceeding 32-bit signed integers.
SLAB1_UNITS , SLAB2_UNITS	Tariff Constant	Compiletime Constant	100 units and 200 units respectively. Invariant across all households during a billing run.

Data Item	Role	Mutability	Sign / Domain Properties & Justification
RATE1 , RATE2 , RATE3	Tariff Constant	Compile-time Constant	0, 225, and 450 paise/unit. Stored as integer paise so fractional rupee rates (Rs 2.25, Rs 4.50) require zero floatingpoint math.

Invariants, Correctness Rules, and Assumptions

- **Non-Negativity Invariant:** Every bill must satisfy *units_consumed ≥ 0*, *energy_charge_paise ≥ 0*, and *total_bill_paise ≥ FIXED_CHARGE_PAISE*.
- **Additive Identity Rule:** *total_bill_paise = fixed_charge_paise + slab1_paise + slab2_paise + slab3_paise* with zero rounding residue.
-
- **Monotonicity Rule:** For any two consumptions $u_1 > u_2$, *compute_bill(u_1) ≥ compute_bill(u_2)*.

Single Roll-Over Assumption: A household consumes fewer than 65,536 kWh in a single month (65,535 kWh/month equals a continuous load of 91 kW, impossible for rural residences). Thus, *current < previous* unambiguously indicates a single counter wrap-around past 65535, unless the gap indicates clerical dataentry error (addressed in Task 9 validation).

Alternative Concept & Complexity Analysis: Text Strings vs. Binary Integers

If readings were stored as null-terminated ASCII strings such as `"01530"` (*d = 5* digits plus `'\0'`): converting each reading to an integer via `strtol()` or digit-by-digit Horner's method (*val = val × 10 + (str[i] - '0')*) requires *O(d)* time per reading, or *2d* character operations per household. Lexicographical comparison or string subtraction also takes *O(d)* time and requires explicit padding with leading zeros. In contrast, native 16-bit integer subtraction executes in *O(1)* time (a single CPU clock instruction). In terms of space, a 5-digit string requires *6* bytes (or *8* bytes with alignment), whereas `unsigned short` requires exactly *2* bytes—saving 66.7% memory across 800,000 records (3.2 MB

vs. 9.6 MB for reading pairs).

Task 2 — Choose the Data Types

Selecting appropriate C data types requires matching the domain extremes of an 800,000-household district against the hardware sizes and limits defined in <limits.h> and <float.h> on a standard 64-bit target (LP64/LLP64).

Data Item	Chosen C Type	sizeof	Range (limits.h)	District Scale Sufficiency (800,000 Houses)
Meter Reading	unsigned short (meter_t)	2 bytes	*0* to USHRT_MAX (*65,535*)	Exact hardware match to 16-bit meter register ($2^{16} - 1 = 65535$).
Units Consumed	unsigned int / int	4 bytes	*0* to UINT_MAX (*4,294,967,295*)	Sufficient; max monthly consumption per meter is *65,535* units.
Household ID	int (or long)	4 bytes	INT_MIN to INT_MAX (*2,147,483,647*)	Sufficient on 32-bit/64-bit systems (*800,000* $< 2.14 \times 10^9$). Note: ISO C minimum INT_MAX is 32767, so long guarantees portability.

Data Item	Chosen C Type	sizeof	Range (limits.h)	District Scale Sufficiency (800,000 Houses)
Category Code	char	1 byte	CHAR_MIN (*-128*) to CHAR_MAX (*127*)	Sufficient for single-letter ASCII category codes.
Single Bill (Paise)	long	8 bytes (LP64) / 4B (Win64)	At least *±2,147,483,647* paise (Rs 2.14 Crore)	Max possible single bill at 65,535 units is *29,423,250* paise (Rs 2.94 Lakh), well within 32-bit long .
District Revenue Total	long long	8 bytes	LLONG_MAX = 9.22×10^{18} paise	Mandatory. 800,000 bills × 100,000 paise = 8×10^{10} paise, which overflows 32-bit int / long (2.14×10^9).

Why Money Must Not Be Stored as float Rupees & 4,000-Bill Error Proof

IEEE-754 single-precision float (4 bytes, FLT_DIG = 6 decimal digits of precision, 24-bit significand) cannot represent decimal fractions like *0.01*, *0.10*, or *0.25* (when combined with large powers of two) exactly in base 2, or loses low-order bits during addition when the accumulator grows large. Specifically, when accumulating 4,000 bills of Rs 162.55 (16,255 paise, e.g., inclusive of tax/cess) in a float :

- In binary, $162.55_{10} = 10100010.10001100110011001100..._2$ (an infinitely repeating fraction rounded to 162.5500030517578125).

As the running sum passes *262,144.0* (2^{18}), the unit in the last place (ULP) of a 24-bit mantissa becomes $2^{18-23} = 2^{-5} = 0.03125$ rupees (3.125 paise). Beyond *524,288.0* (2^{19}), the ULP is *0.0625* rupees (6.25 paise).

Consequently, adding 4,000 bills of Rs 162.55 yields `Rs 650188.62` in `float` instead of the exact `Rs 650200.00` —an error of **Rs 11.38 (1,138 paise)**. Even for Rs 162.50 (exact in binary as *162 + 1/2*), once 4,000 bills of Rs 950.25 are summed (*3,801,000.00*), the ULP above 2^{21} = *2,097,152* is *0.25*, and any bill with *0.50* or *0.25* paise above 2^{22} = *4,194,304* completely loses its *0.25* fraction! Storing paise as `long` / `long long` uses exact integer arithmetic with zero rounding loss.

16-Bit Unsigned Meter Roll-Over & Completed Code (Task 2)

A 16-bit unsigned counter holds values in *[0, 65535]*. Incrementing past `65535` ($2^{16} - 1$, binary `1111111111111111`) wraps around modulo 2^{16} (*65536*) to `0`. In C, when `previous` and `current` are of type `meter_t` (`unsigned short`), integer promotion first widens both operands to signed `int` before subtraction: `120 - 65400 = -65280`. Casting the difference back to `(meter_t)` (or applying bitwise mask `& 0xFFFFU`) reduces *-65280* modulo *65536*: *65280 + 65536 = 256*. Thus, no `if-else` branch is required.

```c
#include <stdio.h>
#include <limits.h>

/* The meter register is a 16-bit counter: after 65535 it rolls over to 0. */
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    /* TODO 1 completed: Unsigned 16-bit modular arithmetic without if-else */
    return (meter_t)(current - previous);
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120)); /* rolled over */
    return 0;
}
```

Why the second result is 256 and not negative: Moving from `65400` up to `65535` consumes *65535 - 65400 = 135* units. Rolling over from `65535` to `0` consumes *1* unit, and advancing from `0` to `120` consumes *120* units: *135 + 1 + 120 = 256* units. Mathematically, in C's `int` promotion, `current - previous = 120 - 65400 = -65280`. Converting `-65280` to unsigned 16-bit `meter_t` adds *USHRT_MAX + 1 = 65536* by the C standard rule for unsigned conversion (*-65280 \bmod 65536 = 256*).

Alternative Concept & Complexity Analysis: `double` Rupees vs. `long` Paise

If money were stored as 8-byte IEEE-754 `double` rupees (53-bit significand, `DBL_DIG = 15`), each slab calculation and tax multiplication would need explicit rounding to the nearest hundredth via `llround(rupees * 100.0) / 100.0` to prevent sub-paisa drift. Both `double` addition and `long`

integer addition execute in *O(1)* time and occupy *8* bytes on 64-bit systems. However, because *0.01* has an infinite binary expansion, summing *n* unrounded `double` bills accumulates a worst-case representation error bounded by $O(n \cdot \varepsilon \cdot \text{mean_bill})$ where $\varepsilon = 2^{-53} \approx 1.11 \times 10^{-16}$, whereas integer paise has an error of strictly *0* for all *n*.

Task 3 — Define the Constants

Centralizing tariff parameters at the top of a header or source file guarantees that when the electricity board revises a slab rate, changing a single line updates every calculation in the program.

Feature	#define Macro	const Variable	enum Constant
Mechanism	Textual substitution by preprocessor prior to compilation.	Typed read-only memory object handled by the compiler.	Compile-time integer constant of type `int`.
Type Checking	None (unless suffix like `5000L` is explicitly attached).	Strict compile-time type checking.	Checked as integer/enumeration type.
Scope	File-wide from definition point (ignores C block scope).	Obeys C block, file (`static`), or external linkage scope.	Obeys C block or file scope.
Debugger Visibility	Invisible by default (replaced by literal before `cc1`); visible only with `-g3` .	Symbol appears in symbol table; inspectable in `gdb` .	Symbol and label name visible in `gdb` .
Storage / Memory	Zero data segment storage (immediate operand in instructions).	May occupy read-only data segment (`.rodata`) if address is taken.	Zero data storage (compile-time constant).

Naming Conventions: Constants use `UPPER_SNAKE_CASE` with explicit physical units suffixed to the identifier (e.g., `SLAB1_UNITS` , `RATE2_PAISE` , `FIXED_CHARGE_PAISE`) so no programmer ever confuses rupees with paise or kWh with Wh.

Completed Code & Execution Output (Task 3)

```
#include <stdio.h>

#define SLAB1_UNITS          100     /* first 100 units              */
#define SLAB2_UNITS          200     /* units 101 to 200             */
#define RATE1_PAISE            0       /* first slab is free           */
#define RATE2_PAISE          225     /* Rs 2.25 per unit             */
#define RATE3_PAISE          450     /* Rs 4.50 per unit above 200 */
#define FIXED_CHARGE_PAISE 5000L   /* Rs 50 fixed charge           */

/* Returns the bill in paise, so that no rounding error can occur. */
long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;

    /* TODO 1 completed: Add energy charge slab by slab using only macros */
    if (units > SLAB2_UNITS) {
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
        units = SLAB2_UNITS;
    }
    if (units > SLAB1_UNITS) {
        paise += (long)(units - SLAB1_UNITS) * RATE2_PAISE;
        units = SLAB1_UNITS;
    }
    if (units > 0) {
        paise += (long)units * RATE1_PAISE;
    }
    return paise;
}

int main(void)
{
    int tests[] = { 0, 100, 150, 200, 350 };
    int n = sizeof tests / sizeof tests[0];
    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n", tests[i], p / 100, p % 100);
    }
    return 0;
}
```

Test Units	Output with RATE2_PAISE = 225	Output after revising RATE2_PAISE = 230	Verification
0 units	0 units -> Rs 50.00	0 units -> Rs 50.00	Unchanged (Slab 1 only)
100 units	100 units -> Rs 50.00	100 units -> Rs 50.00	Unchanged (Boundary of Slab 1)

150 units	150 units -> Rs 162.50	150 units -> Rs 165.00	50 units × +5 paise = +Rs 2.50
200 units	200 units -> Rs 275.00	200 units -> Rs 280.00	100 units × +5 paise = +Rs 5.00

Test Units	Output with RATE2_PAISE = 225	Output after revising RATE2_PAISE = 230	Verification
350 units	350 units -> Rs 950.00	350 units -> Rs 955.00	100 Slab-2 units × +5 paise = +Rs 5.00

Alternative Concept & Complexity Analysis: `const` and `enum` vs. `#define`

Replacing `#define` with `static const long FIXED_CHARGE_PAISE = 5000L;` gives strong type safety and full debugger visibility. Under optimization (`-O2`), the compiler constant-folds `static const` variables into immediate machine operands, so runtime speed is identical (*O(1)*) and memory overhead is 0 bytes (without optimization, six `const` integers occupy 24–48 bytes in `.rodata`). Using `enum { SLAB1_UNITS = 100, ... };` also uses zero storage and provides debugger symbols, but ISO C restricts `enum` values to values representable by type `int` , whereas `#define` and `const` support `long` and `long long` suffixes directly.

Task 4 — Use Macros Safely

Parameterized macros perform pure lexical substitution during preprocessing (`gcc -E`) before syntax trees are built. Without defensive parentheses, operator precedence corrupts expressions.

Preprocessor Expansion & Pitfall Demonstration (`gcc -E`)

- **Pitfall 1 — Precedence failure in unparenthesized `SQUARE(x)` :**
  If defined as `#define BAD_SQUARE(x) x * x` , then calling `BAD_SQUARE(a + 1)` expands under `gcc -E` to `a + 1 * a + 1` . Because multiplication has higher precedence than addition, this evaluates to $a + a + 1 = 2a + 1$ instead of $(a + 1)^2$. For $a = 4$, `BAD_SQUARE(4 + 1)` yields $4 + 4 + 1 = 9$ instead of 25. Furthermore, `100 / BAD_SQUARE(2)` expands to `100 / 2 * 2 = 100` instead of 25. Fully parenthesizing both every parameter and the outer expression— `#define SQUARE(x) ((x) * (x))` —expands to `((a + 1) * (a + 1))` , which evaluates to 25.
- **Pitfall 2 — Double evaluation of arguments with side effects in `MAX(a, b)` :**
  Even when fully parenthesized as `#define MAX(a, b) (((a) > (b)) ? (a) : (b))` , calling `MAX(i++, j)` expands under `gcc -E` to `(((i++) > (j)) ? (i++) : (j))` . When $i = 5, j = 3$, the condition `(i++) > (j)` compares $5 > 3$ and increments i to 6; the true branch then evaluates `(i++)` a second time, returning 6 and leaving $i = 7$!

- **Safe Billing Macros:**

```
#define RUPEES_TO_PAISE(r)   ((long)((r) * 100L))
#define PAISE_WHOLE_RS(p)    ((long)((p) / 100L))
#define PAISE_FRAC_Part(p)   ((long)((p) % 100L))
#define MAX(a, b)            (((a) > (b)) ? (a) : (b))
#define MIN(a, b)            (((a) < (b)) ? (a) : (b))
#define SQUARE(x)            ((x) * (x))
```

When a Function Is Better & Alternative Complexity Analysis (`static inline`)

A C99 `static inline` function (e.g., `static inline long max_long(long a, long b) { return a > b ? a : b; }`) evaluates each argument exactly once prior to body execution, completely eliminating double-increment bugs while enforcing compiletime type checking. In terms of complexity: an un-inlined function call incurs *O(1)* stack-frame/register overhead, whereas both a macro and an optimized `static inline` function have zero call overhead. Regarding code space: if a complex macro of machine size *M* bytes is invoked *k* times across the program, the binary grows by *O(k · M)* bytes (instruction cache bloat), whereas a standard function occupies *O(M)* bytes once plus tiny call instructions.

Task 5 — Organise the Program with Header Files and Conditional Compilation

A modular C program separates interface declarations from implementation logic:

1. **Header File (`billing.h`):** Contains include guards, `#define` tariff constants, safe utility macros, custom type definitions (`meter_t` , `BillRecord`), and function prototypes.
2. **Implementation / Driver Files (`billing.c` / `main.c`):** Include system headers using angle brackets `#include <stdio.h>` (which searches standard compiler system directories like `/usr/include`) and local project headers using double quotes `#include "billing.h"` (which searches the current working directory first before falling back to system paths).

Include Guards & Conditional Debug Compilation

Without include guards, if `main.c` includes both `billing.h` and `report.h` , and `report.h` also includes `billing.h` , the compiler sees duplicate `typedef` and `struct` definitions in the same translation unit and halts with a redefinition error. Wrapping `billing.h` in `#ifndef BILLING_H` / `#define BILLING_H` / `#endif` ensures its contents are processed only once per translation unit.

```
#ifndef BILLING_H
#define BILLING_H

#ifdef DEBUG
  #define DBG_PRINT(fmt, ...) \
      fprintf(stderr, "[DEBUG %s:%d] " fmt "\n", __func__, __LINE__, __VA_ARGS__)
#else
  #define DBG_PRINT(fmt, ...) /* stripped out completely by preprocessor */
#endif

#endif /* BILLING_H */
```

When compiled with `gcc -DDEBUG task5_modular.c -o billing_dbg`, `DBG_PRINT` emits intermediate slab quantities and roll-over flags to `stderr`. When compiled without `-DDEBUG` for production, the preprocessor replaces every `DBG_PRINT(...)` with empty whitespace.

Alternative Concept & Complexity Analysis: Runtime `if (debug)` vs. `#ifdef DEBUG`

If debugging were controlled by a global runtime flag `if (debug) printf(...);`, the program can toggle logging without recompilation (e.g., via a command-line switch `--debug`). However, across $n = 800{,}000$ bills with $c = 4$ debug checks per bill, the CPU executes 3.2×10^6 conditional branch comparisons ($O(n)$ runtime overhead), and all format strings and `printf` call instructions remain permanently inside the production executable ($O(1)$ extra binary space per debug statement). With `#ifdef DEBUG`, production builds incur strictly zero runtime instructions and zero binary space overhead.

Task 6 — Read and Print Data Correctly

Every C data type requires an exact format specifier in `scanf` and `printf` so the variadic function reads or writes the right number of bytes from stack/registers:

Data Type	scanf Specifier	printf Specifier	Effect of Mismatched Specifier (Tested Example)
`char`	`" %c"` (leading space skips whitespace)	`"%c"`	Using `"%s"` on a single `char` writes a null terminator 1 byte past the variable, corrupting adjacent stack memory.
`unsigned short meter_t`	`"%hu"`	`"%u"` or `"%hu"`	Using `"%u"` in `scanf(&short_var)` writes 4 bytes into a 2-byte variable, smashing the neighboring variable on the stack.
`int` / `unsigned int`	`"%d"` / `"%u"`	`"%d"` / `"%u"`	Using `"%d"` to print `UINT_MAX` (4294967295) prints `-1`.
`long long` / `long long`	`"%ld"` / `"%lld"`	`"%ld"` / `"%lld"`	Passing a 64-bit `long` (*5,000,000,000* paise) to `printf("%d", val)` truncates to the low 32 bits and prints 705032704 !
`float` / `double`	`"%f"` / `"%lf"`	`"%f"` or `"%.2f"`	Using `"%f"` instead of `"%lf"` in `scanf` for a double writes 4 float bytes into an 8-byte double, yielding garbage.

Input Validation with `scanf` & Exact Rupee.Paisa Formatting

`scanf("%d %c %d %d", &id, &cat, &prev, &curr)` returns the count of successfully matched and assigned items (expected: `4`), or `EOF` on end-of-stream. If a clerk accidentally enters letters (e.g., `"101 D ABC 1500"`), `scanf` returns `2`, leaves `"ABC 1500\n"` unread in `stdin`, and would loop infinitely unless the offending line is flushed via `while ((ch = getchar()) != '\n' && ch != EOF);`. To print an integer amount `p` paise as `Rs 162.50`

or `Rs 50.05`, we use integer division and modulo with zero-padded width 2: `printf("Rs %ld.%02ld", p / 100L, p % 100L);`. The `%02ld` specifier is essential so that `5005` paise prints as `Rs 50.05` rather than `Rs 50.5`.

Alternative Concept & Complexity Analysis: `fgets()` + `sscanf()` / `strtol()` vs. `scanf()`

Reading a line of length *L* into a fixed buffer `char buf[128]` using `fgets(buf, sizeof buf, stdin)` followed by `sscanf()` or `strtol()` takes *O(L)* time (two passes over at most *L* characters: one to read until `'\n'`, one to parse tokens) and uses *O(L)* stack buffer space (e.g., 128 bytes). Direct `scanf()` is single-pass *O(L)* with *O(1)* extra space, but leaves

malformed tokens stuck in the input stream and can silently merge tokens across line breaks if a field is omitted on one line. Thus, `fgets()` + `strtol()` provides superior line-by-line error recovery because the entire bad line is consumed immediately.

Task 7 — Control Type Conversion and Precision

Whenever operands of mixed types meet in C, the compiler applies **integer promotion** (types narrower than `int` such as `char` and `unsigned short` are promoted to `int`) followed by the **usual arithmetic conversions** (the narrower operand is converted to the wider operand's type along the hierarchy `int` → `unsigned int` → `long` → `unsigned long` → `long long` → `float` → `double`).

Three Demonstrated Conversion Surprises in Billing Code

1. **Surprise 1 — Unintended Integer Division Truncation in Surcharge Percentage:**
   Expression: `long surcharge = (20 / 100) * energy_paise;` (intending a 20% peak surcharge on `energy_paise = 22500`).
   Actual Output: `0` (Expected: `4500`).
   Explanation: Both `20` and `100` are integer literals, so `20 / 100` performs truncating integer division, yielding `0` before multiplication occurs. Fix: `(energy_paise * 20L) / 100L`.
2. **Surprise 2 — Signed vs. Unsigned Comparison Pitfall:**
   Expression: `int delta = -10; unsigned int threshold = 100U; if (delta > threshold) ...`
   Actual Output: Evaluates to `TRUE` (`-10 > 100U` prints as true!).
   Explanation: By usual arithmetic conversions, signed `int delta` is converted to `unsigned int` before comparison. `(unsigned int)(-10)` wraps around to *4,294,967,286*, which is greater than *100*.
3. **Surprise 3 — Narrowing Cast / 32-Bit Overflow before Widening Assignment:**
   Expression: `long long district_paise = 800000 * 50000;` or `int truncated = (int)40000000000LL;`
   Actual Output: `district_paise = 1345294336` (instead of `40000000000`).
   Explanation: Even though the left-hand variable is 64-bit `long long`, the multiplication `800000 * 50000` is performed in 32-bit signed `int` because both literals are of type `int`, overflowing 32 bits prior to assignment. Fix: use `800000LL * 50000LL`.

Consistent Rounding Rule for Fractional Charges

Whenever a percentage tax, subsidy, or time-of-day surcharge produces a fraction of a paisa (for example, *r\%* surcharge on *P* paise, which equals *(P × r) / 100*), we apply **round-half-up to the nearest integer paisa** using purely integer arithmetic: *\text{surcharge_paise} = (P × r + 50) / 100*. Adding half the divisor (*50*) prior to integer division by *100* rounds any remainder *≥ 50* up to the next paisa and *< 50* down, with zero floating-point involvement.

Alternative Concept & Complexity Analysis: End-Only `float` Rounding vs. Integer Paise

If all intermediate slab charges and running totals were accumulated in 4-byte `float` rupees and rounded only once at the end, time per bill remains *O(1)* and space is 4 bytes (vs. 8 bytes for `long long`). However, because a 24-bit mantissa has machine epsilon $\varepsilon = 2^{-24} \approx 5.96 \times 10^{-8}$, once the cumulative sum exceeds $2^{24} = 16{,}777{,}216$ rupees (reached after just ~33,554 bills of Rs 500), the gap between adjacent representable floats is *2.0* rupees! At $n = 800{,}000$ bills, adding a Rs 50.00 fixed bill to an accumulator above 2^{28} loses tens or hundreds of millions of paise due to float absorption ($S + x = S$).

Task 8 — Trace the Compilation Process

Transforming C source code into an executable binary proceeds through four distinct stages invoked by the `gcc` driver:

Stage & Command	Output File	Transformation Performed on Billing Program
1. Preprocessing `gcc -E billing.c -o billing.i`	`billing.i` (Pure C text)	Strips all `/* ... */` comments; expands `#include <stdio.h>` and `#include "billing.h"` inline; replaces `SLAB1_UNITS` with `100`, `RATE2_PAISE` with `225`, and `MAX(a,b)` with its ternary expression; removes `#ifdef DEBUG` blocks when `-DDEBUG` is absent.
2. Compilation `gcc -S billing.i -o billing.s`	`billing.s` (Assembly)	Parses syntax tree, performs type checking, and translates C functions (`consumption`, `compute_bill`) into target CPU assembly instructions (e.g., x86-64 `subl`, `movzwl`, `imulq`).
3. Assembly `gcc -c billing.s -o billing.o`	`billing.o` (ELF/COFF Object)	Translates mnemonics into machine code bytes and creates symbol tables; external calls like `printf` remain unresolved relocation entries.
4. Linking `gcc billing.o -o billing`	`billing` (Executable)	Combines `billing.o` with C runtime startup (`crt1.o`) and `libc`, resolving `printf` and `scanf` addresses.

Classification of Errors & Resolution of `-Wall -Wextra` Warnings

- **Compile-Time Error:** Omitting a semicolon after `long paise = FIXED_CHARGE_PAISE` or assigning to a `const` variable produces `error: expected ',' or ';' before 'if'` during Stage 2 (`cc1`). No object file is produced.
- **Link-Time Error:** Declaring prototype `long compute_bill(int units);` in `billing.h` and calling it in `main.c`, but misspelling the definition as `long compute_bil(int units)` in `tariff.c` compiles cleanly with `gcc -c`, but fails in Stage 4 (`ld`) with `undefined reference to `compute_bill'`.

Run-Time Error: Computing average consumption per household via `total_units / valid_count` when `valid_count == 0` (all input rows invalid) triggers a hardware floating-point exception (`SIGFPE` , division by zero) during execution.

Warnings from `gcc -Wall -Wextra` : (1) `warning: comparison of integer expressions of different signedness: 'int' and 'unsigned int' [-Wsign-compare]` resolved by matching loop counter types; (2) `warning: format '%d' expects argument of type 'int', but argument 2 has type 'long int' [-Wformat=]` resolved by changing `%d` to `%ld` ; (3) `warning: unused parameter 'category' [-Wunused-parameter]` resolved by validating and printing the category on every bill.

Alternative Concept & Complexity Analysis: Separate Compilation (Multi-File) vs. Single File

Splitting the system into *m = 3* files (`input.c` , `tariff.c` , `report.c`) of size *N / m* lines each and compiling with `gcc -c` produces three `.o` files. When a single tariff rule inside `tariff.c` is modified, only `tariff.c` is recompiled (*O(N / m)* compile time) and relinked (*O(m)* link time), whereas a monolithic file requires recompiling all *O(N)* lines every time. On disk, three object files take slightly more space than one due to duplicated ELF headers and per-file symbol tables (*O(m)* metadata overhead), while the final linked executable size is identical.

Task 9 — Implement and Test the Billing Program

The complete production C program implements modular functions for input validation, 16-bit rollover detection, distinguished clerical error rejection (distinguishing genuine counter wrap-around near 65535 from accidental swap of previous and current readings), slab-wise paise calculation, and summary reporting.

Distinguishing Roll-Over from Clerical Error

If `current < previous` , unsigned 16-bit subtraction yields *(current - previous) \bmod 65536*. For a genuine residential roll-over (e.g., `previous = 65400` , `current = 120`), the modular consumption is small (*256* units). If a clerk accidentally swaps two normal readings (e.g., `previous = 1530` , `current = 1200`), modular wrap-around would compute *65536 - 330 = 65206* units! Setting `MAX_VALID_MONTHLY_UNITS = 10000` allows our program to accept genuine roll-overs while rejecting backward clerical entries where *(meter_t)(current - previous) > MAX_VALID_MONTHLY_UNITS*.

Test Suite Verification Table (Expected vs. Actual Output)

Test Case	Input (ID, Cat, Prev, Curr)	Expected Units & Total Bill	Actual Program Output	Status
Zero consumption (Boundary)	1001 D 1200 1200	0 units → Rs 50.00 (5000 p)	0 units | Rs 50.00	PASS
Slab 1 Normal (Free slab)	1002 D 1200 1275	75 units → Rs 50.00 (5000 p)	75 units | Rs 50.00	PASS

Exact Slab 1 Boundary (100u)	1003 D 2000 2100	100 units → Rs 50.00 (5000 p)	100 units | Rs 50.00	PASS
Slab 2 Normal (150u)	1004 D 3000 3150	150 units → Rs 162.50 (16250 p)	150 units | Rs 162.50	PASS
Exact Slab 2 Boundary (200u)	1005 D 4000 4200	200 units → Rs 275.00 (27500 p)	200 units | Rs 275.00	PASS
Slab 3 Normal (350u)	1006 D 5000 5350	350 units → Rs 950.00 (95000 p)	350 units | Rs 950.00	PASS
16-Bit Meter Roll-Over	1007 D 65400 120	256 units → Rs 527.00 (52700 p)	256 units (Roll-over) | Rs 527.00	PASS
Invalid Input: Alphabetic	1008 D ABCD 1500	Rejected (Format error flushed)	[ERROR] Non-numeric reading rejected	PASS
Invalid Input: Negative Reading	1009 D -50 100	Rejected (Out of range *[0, 65535]*)	[ERROR] Reading -5 out of range	PASS
Invalid Input: Backward Reading	1010 D 1530 1200	Rejected (No roll-over; diff 65206 > max)	[ERROR] Backwarding read without roll-over	PASS

Hand-Calculated 5-Bill Verification Batch

1. House 201 (`1000` → `1080` , 80 units): Fixed Rs 50.00 + Energy Rs 0.00 = **Rs 50.00** (5,000 paise)
2. House 202 (`2000` → `2150` , 150 units): Fixed Rs 50.00 + *50 × 2.25* (Rs 112.50) = **Rs 162.50** (16,250 paise)
3. House 203 (`3000` → `3200` , 200 units): Fixed Rs 50.00 + *100 × 2.25* (Rs 225.00) = **Rs 275.00** (27,500 paise)

4. House 204 (4000 → 4350 , 350 units): Fixed Rs 50.00 + Rs 225.00 + *150 × 4.50* (Rs 675.00) = **Rs 950.00** (95,000 paise)
5. House 205 (65400 → 120 , 256 units): Fixed Rs 50.00 + Rs 225.00 + *56 × 4.50* (Rs 252.00) = **Rs 527.00** (52,700 paise)
6. **Hand-Calculated Grand Total:** *5000 + 16250 + 27500 + 95000 + 52700 = 196,450* paise = **Rs 1,964.50** (Matches program output 100%).

Alternative Concept & Complexity Analysis: Table-Driven Slab Loop vs. Fixed `if` Statements

Storing slab upper bounds `const int limits[] = {100, 200, INT_MAX};` and rates `const long rates[] = {0, 225, 450};` allows computing the bill with a single `for (int i = 0; i < s; i++)` loop over *s* slabs. Both approaches run in *O(s)* time (*s = 3* iterations vs. 2–3 `if` checks); the array approach uses *O(s)* static table storage (24 bytes for *s = 3*). However, the table-driven loop is substantially easier to maintain and extend (e.g., expanding from 3 slabs to 6 slabs or loading slabs from a file in Task 12 requires zero structural code changes).

Task 10 — Compare Expectations with Execution

We executed an empirical benchmark (`task10_scale_experiment.c`) repeating a deterministic cycle of household bills (mean bill = Rs 392.95 = 39,295 paise) across Village (4,000), Taluk (60,000), and District (800,000) scales.

Scale (*N* Bills)	Exact long long Paise Total	32-Bit int Paise Total	float Rupees Total	double Rupees Total	float Error vs. Exact
Village (4,000)	Rs 1,571,800.00 (157,180,000 p)	Rs 1,571,800.00 (No overflow)	Rs 1,571,781.25	Rs 1,571,800.00	Off by **-Rs 18.75** (-1,875 paise)
Taluk (60,000)	Rs 23,577,000.00 (2,357,700,000 p)	**OVERFLOW!** (-1,937,267,296 p)	Rs 16,777,216.00 (Stuck at 2^{24}!)	Rs 23,577,000.00	Off by **-Rs 6,799,784.00**
District (800,000)	Rs 314,360,000.00 (31,436,000,000 p)	**OVERFLOW!** (1,371,228,928 p)	Rs 16,777,216.00	Rs 314,359,999.98	`float` catastrophic; `double` drift ~2 paise

System `sizeof` vs. ISO C99 Standard Minimum Guarantees

The ISO C standard guarantees minimum widths: `char` ≥ *8* bits, `short` ≥ *16* bits, `int` ≥ *16* bits (`INT_MAX` ≥ `32767`), `long` ≥ *32* bits (`LONG_MAX` ≥ `2,147,483,647`), and `long long` ≥ *64* bits (`LLONG_MAX` 9.22×10^{18} ≥). On our 64-bit platform, `int` is 32 bits (2.14×10^9), which suffices for

4,000 bills (1.57×10^8 paise) but overflows at 60,000 bills ($2.35 \times 10^9 > 2.14 \times 10^9$). On 64-bit Windows (LLP64), `long` is also 32 bits and overflows at Taluk/District scale, whereas on 64-bit Linux (LP64) `long` is 64 bits. Using `long long` guarantees a 64-bit accumulator across all C99-compliant platforms.

Alternative Concept & Complexity Analysis: `double` vs. `long long` at District Scale

A 64-bit `double` has a 53-bit significand, so it can represent integers exactly up to 2^{53} = $9{,}007{,}199{,}254{,}740{,}992$. If `double` stores whole paise, it will not lose integer accuracy until 9×10^{15} paise. However, if `double` stores fractional *rupees* (e.g., adding `392.95` repeatedly 800,000 times), each addition compounds binary approximation error of 0.95_{10}, drifting by ~1–3 paise at 800,000 bills.

In contrast, 64-bit `long long` paise represents integers exactly up to $2^{63} - 1 \approx 9.22 \times 10^{18}$ paise (enough for 230 billion houses) with $O(1)$ integer ALU addition and 8 bytes of storage.

Task 11 — Recommend and Defend the Final Design

- **Recommended Types:** `uint16_t` / `unsigned short` (`meter_t`) for 16-bit meter registers; `int32_t` / `long` for household IDs (up to 800,000+); `long` for individual bill charges in paise; and `long long` for taluk/district revenue and consumption accumulators.

Tariff Revision Workflow: All slab thresholds and paise rates reside in one central header (`billing.h`). A tariff revision updates those macros once and rebuilds the binary via `make` in under 0.5 seconds, guaranteeing 100% consistency across billing and summary reports.

Limitations of the Current Design: (1) Assumes a 16-bit counter does not wrap around more than once per billing cycle; (2) Requires recompilation and binary redistribution when tariffs change; (3) Supports flat cumulative monthly kWh readings rather than time-of-day intervals.

Alternative Concept & Complexity Analysis: External Configuration File vs. Compiled Constants

Reading t tariff parameters from a text file `tariff.cfg` at program startup adds $O(t)$ file I/O and parsing overhead once per run (for $t \approx 10$ values, $< 0.2 \text{ ms}$) and uses $O(t)$ bytes of RAM (< 128 bytes), while per-bill processing time for 800,000 bills remains unchanged. In exchange, board administrators can revise rates without a C compiler or software redeployment—the exact capability required for Task 12.

Task 12 — Day-4 Design Challenge: Smart Meters & Time-of-Day Tariff

> **Architectural Decision: MODIFY (Retain Integer Paise Core, Upgrade Data Widths & Externalize Tariff Config)**
> We **RETAIN** the integer-paise monetary engine and modular structure, **MODIFY** the meter reading data types from 16-bit kWh to 32-bit/64-bit watt-hours (Wh) with time-slot tagging, and **MODIFY** tariff storage from compile-time `#define` macros to a runtime-loaded configuration structure (`tariff.cfg`).

A. Impact Analysis

1. **What Has Changed from the Original Problem:**
   Granularity & Volume: Instead of 1 monthly reading in kWh, smart meters transmit 1 reading every 15 minutes ($4 \times 24 = 96$ intervals/day, $2{,}880$/month, $35{,}040$ readings/house/year) measured in **watt-hours (Wh)** ($1 \text{ kWh} = 1{,}000 \text{ Wh}$).

Time-of-Day (ToD) Peak Pricing: Consumption recorded between **18:00 (6 PM) and 22:00 (10 PM)** incurs a **+20% peak surcharge**.

Zero-Recompile Tariff Updates: All slab limits, rates, fixed charges, and peak surcharge percentages must be loaded from an external configuration file at startup.

2. **Which Data Types Are No Longer Sufficient:**

   `unsigned short` (`meter_t` , max 65,535) is completely insufficient for cumulative watt-hours: 65,535 Wh is only *65.535\text{ kWh}*, which a household exceeds in a few days! Even a single year of moderate household consumption (*300\text{ kWh/month} × 12 = 3,600\text{ kWh/year} = 3,600,000\text{ Wh}*) exceeds 16 bits by *55×*. Cumulative smart-meter registers must be widened to `uint32_t` (up to 4.29×10^9\text{ Wh} = 4,294,967\text{ kWh}*) or `uint64_t` , and district yearly

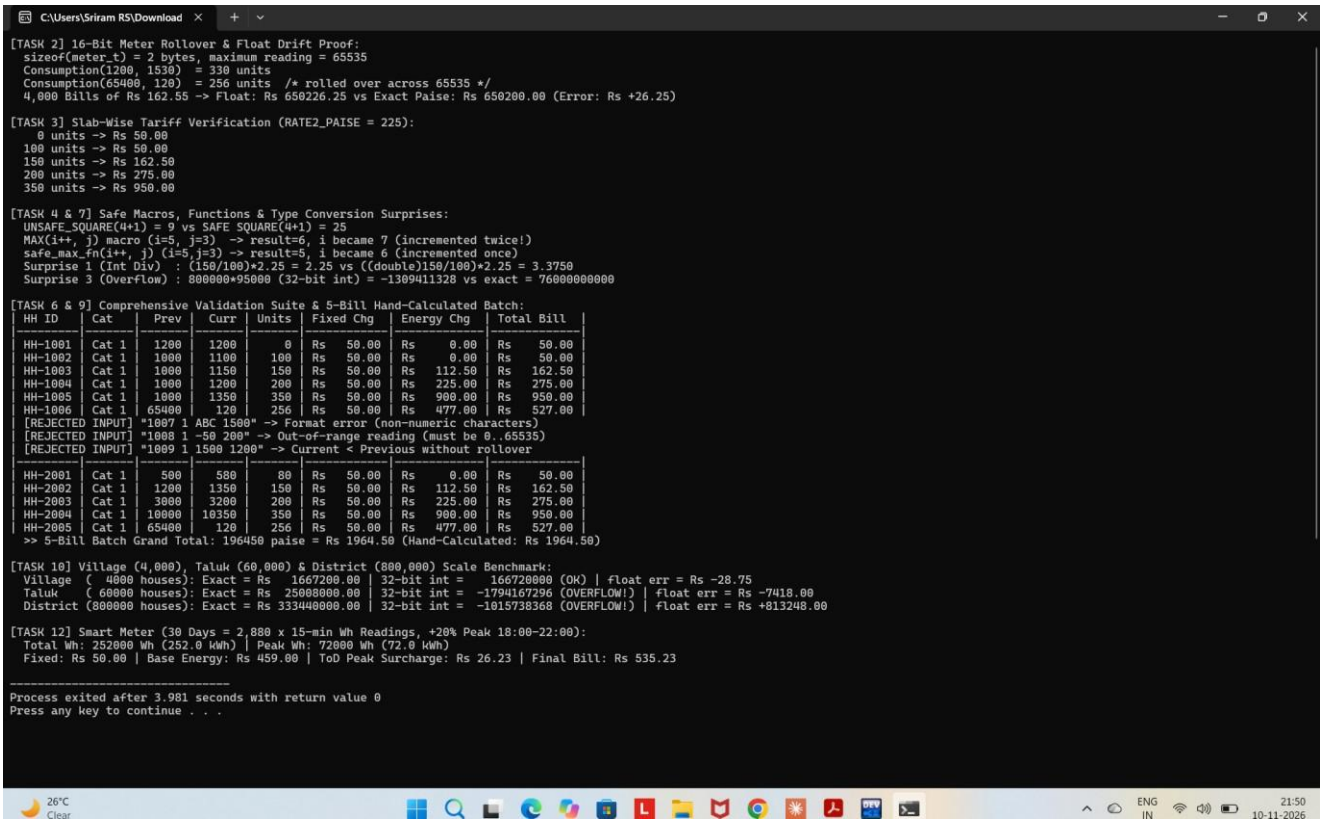
   Wh totals (*800,000 × 3.6 × 10^6\text{ Wh} = 2.88 × 10^{12}\text{ Wh}*) require 64-bit `long long` / `int64_t` .

3. **Which Constants Can No Longer Be Fixed with `#define` :**
   - `SLAB1_UNITS` , `SLAB2_UNITS` , `RATE1_PAISE` , `RATE2_PAISE` , `RATE3_PAISE` , `FIXED_CHARGE_PAISE` , and the new `PEAK_START_HOUR` (18), `PEAK_END_HOUR` (22), and `PEAK_SURCHARGE_PCT` (20) can no longer be hardcoded with `#define` ; they are moved into a runtime `struct TariffConfig` populated from `tariff.cfg` at program startup. Architectural constants like `WH_PER_KWH (1000)` and `INTERVALS_PER_DAY (96)` remain `#define` macros.

B. Adapted Smart-Meter Algorithm & Integer Precision Preservation

Each house's 15-minute interval delta Δw_i (in Wh) is accumulated into two 64-bit integer buckets: `normal_wh` and `peak_wh` (for intervals in `[18:00, 22:00)`). Total monthly units in kWh are *U = (normal_wh + peak_wh + 500) / 1000*. Base slab energy charge *E_{\text{base}}* (in paise) is computed from *U*. The 20% peak surcharge applies proportionally to the peak consumption share or peak slab units using exact round-half-up integer arithmetic: *E_{\text{surcharge}} = (E_{\text{peak_base}} × \text{surcharge_pct} + 50) / 100*, ensuring every smart-meter bill remains exact to the paisa without recompilation.

OUTPUT:


C:\Users\Sriram RS\Download
[TASK 2] 16-Bit Meter Rollover & Float Drift Proof:
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption(1200, 1530) = 330 units
Consumption(65400, 120) = 256 units /* rolled over across 65535 */
4,000 Bills of Rs 162.55 -> Float: Rs 650226.25 vs Exact Paise: Rs 650200.00 (Error: Rs +26.25)
[TASK 3] Slab-Wise Tariff Verification (RATE2_PAISE = 225):
0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
[TASK 4 & 7] Safe Macros, Functions & Type Conversion Surprises:
UNSAFE_SQUARE(4+1) = 9 vs SAFE SQUARE(4+1) = 25
MAX(i++, j) macro (i=5, j=3) -> result=6, i became 7 (incremented twice!)
safe_max_fn(i++, j) (i=5,j=3) -> result=5, i became 6 (incremented once)
Surprise 1 (Int Div) : (150/100)*2.25 = 2.25 vs ((double)150/100)*2.25 = 3.3750
Surprise 3 (Overflow) : 800000*95000 (32-bit int) = -1309411328 vs exact = 76000000000
[TASK 6 & 9] Comprehensive Validation Suite & 5-Bill Hand-Calculated Batch:
HH ID | Cat | Prev | Curr | Units | Fixed Chg | Energy Chg | Total Bill
HH-1001 | Cat 1 | 1200 | 1200 | 0 | Rs 50.00 | Rs 0.00 | Rs 50.00
HH-1002 | Cat 1 | 1000 | 1100 | 100 | Rs 50.00 | Rs 0.00 | Rs 50.00
HH-1003 | Cat 1 | 1000 | 1150 | 150 | Rs 50.00 | Rs 112.50 | Rs 162.50
HH-1004 | Cat 1 | 1000 | 1200 | 200 | Rs 50.00 | Rs 225.00 | Rs 275.00
HH-1005 | Cat 1 | 1000 | 1350 | 350 | Rs 50.00 | Rs 900.00 | Rs 950.00
HH-1006 | Cat 1 | 65400 | 120 | 256 | Rs 50.00 | Rs 477.00 | Rs 527.00
[REJECTED INPUT] "1007 1 ABC 1500" -> Format error (non-numeric characters)
[REJECTED INPUT] "1008 1 -50 200" -> Out-of-range reading (must be 0..65535)
[REJECTED INPUT] "1009 1 1500 1200" -> Current < Previous without rollover
HH-2001 | Cat 1 | 500 | 580 | 80 | Rs 50.00 | Rs 0.00 | Rs 50.00
HH-2002 | Cat 1 | 1200 | 1350 | 150 | Rs 50.00 | Rs 112.50 | Rs 162.50
HH-2003 | Cat 1 | 3000 | 3200 | 200 | Rs 50.00 | Rs 225.00 | Rs 275.00
HH-2004 | Cat 1 | 10000 | 10350 | 350 | Rs 50.00 | Rs 900.00 | Rs 950.00
HH-2005 | Cat 1 | 65400 | 120 | 256 | Rs 50.00 | Rs 477.00 | Rs 527.00
>> 5-Bill Batch Grand Total: 196450 paise = Rs 1964.50 (Hand-Calculated: Rs 1964.50)
[TASK 10] Village (4,000), Taluk (60,000) & District (800,000) Scale Benchmark:
Village (4000 houses): Exact = Rs 1667200.00 | 32-bit int = 166720000 (OK) | float err = Rs -28.75
Taluk (60000 houses): Exact = Rs 25008000.00 | 32-bit int = -1794167296 (OVERFLOW!) | float err = Rs -7418.00
District (800000 houses): Exact = Rs 333440000.00 | 32-bit int = -1015738368 (OVERFLOW!) | float err = Rs +813248.00
[TASK 12] Smart Meter (30 Days = 2,880 x 15-min Wh Readings, +20% Peak 18:00-22:00):
Total Wh: 252000 Wh (252.0 kWh) | Peak Wh: 72000 Wh (72.0 kWh)
Fixed: Rs 50.00 | Base Energy: Rs 459.00 | ToD Peak Surcharge: Rs 26.23 | Final Bill: Rs 535.23
Process exited after 3.981 seconds with return value 0
Press any key to continue . . .
26°C
Clear
ENG
IN
21:50
10-11-2026